Héberger un projet Symfony dans Coolify

Démarche en 2 parties, à suivre dans l'ordre :

- **A)** Adapter le projet Symfony
- **B)** Pousser les modifications sur GitHub
- **C)** MAJ du projet

A) Adapter le projet Symfony

1. `compose.yaml`

Remplacer le contenu de `compose.yaml` (à la racine du projet) par :

```yaml
services:
  php:
    build: .
    restart: unless-stopped
    environment:
      APP_ENV: prod
      APP_SECRET: ${APP_SECRET}
      TRUSTED_PROXIES: ${TRUSTED_PROXIES:-REMOTE_ADDR}
      DATABASE_URL: mysql://${MARIADB_USER:-app}:${MARIADB_PASSWORD}@database:3306/${MARIADB_D
      # Caddy écoute en HTTP sur le port 80 : le HTTPS est géré par Traefik (Coolify)
      SERVER_NAME: :80
    depends_on:
      database:
        condition: service_healthy

  database:
    image: mariadb:${MARIADB_VERSION:-11.4}
    restart: unless-stopped
    environment:
      MARIADB_DATABASE: ${MARIADB_DATABASE:-app}
      MARIADB_USER: ${MARIADB_USER:-app}
      MARIADB_PASSWORD: ${MARIADB_PASSWORD}
      MARIADB_ROOT_PASSWORD: ${MARIADB_ROOT_PASSWORD}
    healthcheck:
      test: ["CMD", "healthcheck.sh", "--connect", "--innodb_initialized"]
      timeout: 5s
      retries: 5
      start_period: 60s
    volumes:
      - database_data:/var/lib/mysql:rw

volumes:
  database_data:
```

Explications :

- `build: .` : l'image du service php est construite à partir du `Dockerfile` situé à la racine du projet (voir l'étape suivante).
- Aucun `ports:` n'est publié : Coolify (via son proxy Traefik) route le trafic vers le port 80 du service php.
- Les `${VARIABLE}` seront renseignées dans Coolify (partie C). `${VARIABLE:-valeur}` définit une valeur par défaut.

2. `Dockerfile`

Créer un fichier `Dockerfile` à la racine du projet, avec ce contenu :

```
FROM dunglas/frankenphp:1-php8.4

RUN install-php-extensions intl opcache pdo_mysql zip

COPY --from=composer:2 /usr/bin/composer /usr/bin/composer

ENV APP_ENV=prod \
    FRANKENPHP_CONFIG="worker ./public/index.php" \
    COMPOSER_ALLOW_SUPERUSER=1

WORKDIR /app
COPY . .

RUN composer install --no-dev --optimize-autoloader --no-scripts --no-interaction \
    && composer dump-env prod \
    && php bin/console importmap:install \
    && php bin/console asset-map:compile \
    && php bin/console cache:warmup \
    && mkdir -p var && chown -R www-data:www-data var

# Migrations puis démarrage du serveur
CMD ["sh", "-c", "php bin/console doctrine:migrations:migrate --no-interaction --allow-no-migr
```

Explications :

- Image de base FrankenPHP avec PHP 8.4 et installation des extensions PHP nécessaires (`pdo_mysql` pour MariaDB).
- `FRANKENPHP_CONFIG="worker ./public/index.php"` active le mode worker de FrankenPHP sur le point d'entrée de Symfony. `symfony/runtime` le détecte automatiquement.
- Le build installe les dépendances sans les paquets de dev, prépare la configuration de production, télécharge les bibliothèques JavaScript (`importmap:install`), compile les assets et réchauffe le cache.
- `importmap:install` est indispensable : `composer install --no-scripts` ne le lance pas, et le dossier `assets/vendor/` est dans le `.gitignore`, donc absent de GitHub. Sans cette commande, `asset-map:compile` échoue avec `The "@hotwired/stimulus" vendor asset is missing`.
- Au démarrage du conteneur, les migrations Doctrine sont exécutées, puis le serveur FrankenPHP est lancé.
- `--allow-no-migration` est nécessaire tant que le projet n'a aucune migration (dossier `src/Migrations/` vide) : sans lui, `doctrine:migrations:migrate` échoue avec `The version "latest" couldn't be reached, there are no registered migrations`, et à cause du `&&` le serveur ne démarre jamais.

3. `.dockerignore`

Créer un fichier nommé exactement `.dockerignore` à la racine du projet (à côté du `Dockerfile`), avec ce contenu :

```
.git/
.idea/
.vscode/
vendor/
var/
node_modules/
.env.local
.env.*.local
docs/
```

Explications :

- Ces fichiers ne sont pas copiés dans l'image par le `COPY . .` du `Dockerfile` : dépendances locales (réinstallées par `composer install`), cache, historique Git, et surtout secrets locaux.
- Ne pas ajouter `.env` : il est nécessaire au build (`composer dump-env prod`).

4. Trusted proxies

Dans `config/packages/framework.yaml`, ajouter sous `framework:` :

```
framework:
    trusted_proxies: '%env(TRUSTED_PROXIES)%'
    trusted_headers: ['x-forwarded-for', 'x-forwarded-host', 'x-forwarded-proto', 'x-forwarde
```

Puis ajouter dans le fichier `.env` (valeur par défaut, committée) :

```
TRUSTED_PROXIES=
```

Explications :

- Traefik (le proxy de Coolify) se place devant l'application et transmet les informations de la requête d'origine dans les en-têtes `X-Forwarded-*`. Sans cette configuration, Symfony les ignore et génère des URL en `http` au lieu de `https`.
- `%env(TRUSTED_PROXIES)%` exige que la variable existe, sinon Symfony s'arrête avec `Environment variable not found: "TRUSTED_PROXIES"`. D'où la ligne vide dans `.env` : en local, il n'y a pas de proxy.
- En production, `TRUSTED_PROXIES` vaut `REMOTE_ADDR` : le `compose.yaml` (étape 1) lui donne cette valeur par défaut, qui l'emporte sur `.env`. `REMOTE_ADDR` est un mot-clé de Symfony qui fait confiance à l'adresse IP qui se connecte directement au conteneur, c'est-à-dire Traefik. Cela convient car le conteneur n'est joignable que via le proxy de Coolify (aucun `ports:` publié).

B) Pousser les modifications sur GitHub

```
git status
git add compose.yaml Dockerfile .dockerignore config/packages/framework.yaml .env
git commit -m "Préparer le déploiement Coolify (FrankenPHP, MariaDB)"
git push origin main
```

Explications :

- Coolify déploie ce qui est sur GitHub : tous les fichiers créés ou modifiés en A doivent être poussés sur la branche `main`.
- `git status` permet de vérifier la liste des fichiers modifiés avant le `git add`.
- Le dépôt doit être **public**, car la ressource Coolify est de type *Public Repository*.
- Ne jamais committer de secrets (`.env.local`, mots de passe) : ils se définissent dans Coolify (partie C).

C) MAJ du projet

Après chaque modification du projet Symfony, suivre ces étapes dans l'ordre.

1. Tester en local

```
php bin/phpunit
php -S localhost:8000 -t public
```

Vérifier que les tests passent et que le site fonctionne avant de pousser.

2. Si les entités Doctrine ont changé : créer la migration

```
php bin/console make:migration
php bin/console doctrine:migrations:migrate
```

- Le fichier de migration généré dans `src/Migrations/` **doit être committé** : il sera exécuté automatiquement au démarrage du conteneur en production (le `CMD` du `Dockerfile` lance `doctrine:migrations:migrate`).
- Ne jamais utiliser `doctrine:schema:update` : les modifications du schéma passent toujours par une migration.
- Les données de la base sont conservées entre les déploiements (volume `database_data`).

3. Si des dépendances ont changé

```
composer require vendor/paquet
php bin/console importmap:require nom-du-paquet-js
```

- `composer require` modifie `composer.json` et `composer.lock` : les deux fichiers doivent être committés.
- `importmap:require` modifie `importmap.php` : ce fichier doit être committé. Le dossier `assets/vendor/` n'est **pas** committé : il est régénéré pendant le build par `importmap:install`.

4. Pousser les modifications sur GitHub

```
git status
git add .
git commit -m "Description de la modification"
git push origin main
```

Vérifier avec `git status` qu'aucun secret (`.env.local`) n'est ajouté.

Attendre la fin du `git push` **avant** de demander le redéploiement : Coolify déploie le dernier commit présent sur GitHub, pas celui de ton poste. Pour vérifier que le push est bien terminé :

```
git status
```

Le message doit indiquer `Your branch is up to date with 'origin/main'`.

5. Demander le redéploiement dans Coolify

Un `git push` ne met **pas** le site à jour tout seul : le site en ligne ne change qu'après un nouveau déploiement dans Coolify.

1. Vérifier que le `git push` est terminé (étape 4 : `Your branch is up to date with 'origin/main'`).
2. Demander à l'enseignant de **redéployer** le projet : lui indiquer l'URL du dépôt dans Github (le code est mis à jour sur la branche `main`).
3. Attendre sa confirmation que le déploiement est terminé.

> Ne pas redemander un déploiement avant d'avoir poussé le code sur GitHub sinon le déploiement est inutile et conserverait l'ancienne version.

6. Vérifier

Ouvrir le site à l'adresse du sous-domaine et tester la modification.